In [1]:
```
import pandas as pd
```

In [2]:
```
df = pd.read_csv("titanic_train.csv")
```

In [3]:
```
df.head()
```

Out[3]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>PassengerId</th> <th>Survived</th> <th>Pclass</th> <th>Name</th> <th>Sex</th> <th>Age</th> <th>SibSp</th> <th>Parch</th> <th>Ticket</th> <th>Fare</th> <th>Cabin</th> <th>Embarked</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171				
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599				
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282				
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803				
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450				

In [4]:

```
df["Age"].isna().sum()
```

Out[4]:
```
np.int64(177)
```

In [5]:
```
df["Age"].median()
```

Out[5]:
```
np.float64(28.0)
```

In [6]:
```
median_age = df["Age"].median()
median_age
```

Out[6]:
```
np.float64(28.0)
```

In [7]:
```
df["Age"] = df["Age"].fillna(median_age)
```

In [8]:
```
df["Age"].isna().sum()
```

Out[8]:
```
np.int64(0)
```

In [9]:
```
df["Sex"] = df["Sex"].map({"male": 0, "fem
```

In [10]:
```
df[["Sex", "Age"]].head()
```

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Sex</th> <th>Age</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0	22.0
1	38.0
1	26.0
1	35.0
0	35.0

In [11]:
```
X = df[["Pclass", "Age", "Sex", "Fare"]]
```

In [12]:
```
y = df["Survived"]
```

In [13]:
```
print(X.shape)
print(y.shape)
```

```
(891, 4)
(891,)
```

In [14]:
```
from sklearn.model_selection import train_
```

In [15]:
```
X_train, X_test, y_train, y_test = train_t
    X, y, test_size=0.30, random_state=42
)
```

In [16]:
```
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))
```

```
Training rows: 623
Testing rows: 268
```

In [17]:
```
from sklearn.tree import DecisionTreeClass
```

In [18]:
```
model = DecisionTreeClassifier(max_depth=3
```

In [19]:
```
model.fit(X_train, y_train)
```

Out[19]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css

set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em;

} /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em;

border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme

color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-

0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999;

/* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding:

.5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table .parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell

containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ }

.features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; }

.features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body> DecisionTreeClassifier(max_depth=3, random_state =42)

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label fitted sk-toggleable__label-arrow"> DecisionTreeClassifier ?Documentation for DecisionTreeClassifieriFitted </label> <details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy max_depth to clipboard" onclick="copyToClipboard('max_depth', this.parentElement.nextElementSibling)" ></button>

max
Non

```
<button type="button" class="copy-paste-icon" aria-label="Copy random_state to clipboard" onclick="copyToClipboard('random_state', this.parentElement.nextElementSibling)" ></button>
```

```
ran

alwa
W
a
fou
Tha

b
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy criterion to clipboard" onclick="copyToClipboard('criterion', this.parentElement.nextElementSibling)" ></button>
```

```
crit
r
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy splitter to clipboard" onclick="copyToClipboard('splitter', this.parentElement.nextElementSibling)" ></button>
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy min_samples_split to clipboard" onclick="copyToClipboard('min_samples_split', this.parentElement.nextElementSibling)" ></button>
```

```
mi
nu
`mir
is
nu
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy min_samples_leaf to clipboard" onclick="copyToClipboard('min_samples_leaf', this.parentElement.nextElementSibling)" ></button>
```

```
m
n
onl
ea

`mi
is
nur
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy min_weight_fraction_leaf to clipboard" onclick="copyToClipboard('min_weight_fraction_leaf', this.parentElement.nextElementSibling)" ></button>
```

```
m
```

```
<button type="button" class="copy-paste-icon" aria-label="Copy max_features to clipboard"
```

```
n
nu
co
```

onclick="copyToClipboard('max_features', this.parentElement.nextElementSibling)" ></button>

c

<button type="button" class="copy-paste-icon" aria-label="Copy max_leaf_nodes to clipboard" onclick="copyToClipboard('max_leaf_nodes', this.parentElement.nextElementSibling)" ></button>

<button type="button" class="copy-paste-icon" aria-label="Copy min_impurity_decrease to clipboard" onclick="copyToClipboard('min_impurity_decrease', this.parentElement.nextElementSibling)" ></button>

r
wi
to th

of<b
chi

Nor
list

<button type="button" class="copy-paste-icon" aria-label="Copy class_weight to clipboard" onclick="copyToClipboard('class_weight', this.parentElement.nextElementSibling)" ></button>

inst
of

n

<button type="button" class="copy-paste-icon" aria-label="Copy ccp_alpha to clipboard" onclick="copyToClipboard('ccp_alpha', this.parentElement.nextElementSibling)" ></button>

ccp

See

<button type="button" class="copy-paste-icon" aria-label="Copy monotonic_cst to clipboard" onclick="copyToClipboard('monotonic_cst', this.parentElement.nextElementSibling)" ></button>

<br

(i.e.

</details>

<details> <summary>Fitted attributes</summary>

<tbody> <th>Name</th> <th>Type</th>

<th>Value</th> </tbody> <tbody></tbody>

classes_ classes_: ndarray of shape (n_classes,) or list of
classes labels (single output problem),
or a list of a
(m

feature_importances_ feature_importances_: ndarray of

The impurity-based feature importances.
important the feature.
The importance of a feature
(normalized)
total reduction of the criterion brough
also
known as the Gini importance [4]_.

Wa
feature importances can be misleading for
high cardi
unique values). See
:func:`sklearn.inspection.permut

feature_names_in_ feature_names_in_: ndarray of shape (`r

Names of features seen during :term:`fit`. Defined o
feature names that are all strings.

max_features_ max_features_: int

The inferred va

n_classes_ n_classes_: int or list of int

The nu
single output problems),
or a list containing the nu
each
output (for mu

n_features_in_ n_features_in_: int

Number of
:term:`fit`.

.

n_outputs_ n_outputs_: int

The number of o

tree_ tree_: Tree instance

The underlying Tree
to
``help(sklearn.tree._tree.Tree)`` for attri
and
:ref:`sphx_glr_auto_examples_tree_plot_unveil_tree_
basic usag

</details>
<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `$paramPrefix$ {text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const

parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `$paramPrefix{paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` " $row " `); $const formattedText = `[\n {rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('✔️', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = '✔️'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';

```
navigator.clipboard.writeText(formattedText)
.then(() => { element.style.display = 'none';
element.parentElement.appendChild(copyMark);
setTimeout(() => { copyMark.remove();
element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }) .catch(err => {
console.error('Failed to copy:', err);
element.style.color = 'orange'; element.innerHTML =
"Failed!"; setTimeout(() => { element.innerHTML =
originalHTML; element.style = originalStyle; }, 1000);
}); return false; } /** * Adapted from Skrub *
https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228
* @returns "light" or "dark" */ function
detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode
theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const
themeNameAttr = body.getAttribute('data-vscode-
theme-name'); if (themeKindAttr &&
themeNameAttr) { const themeKind =
themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if
(themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if
(themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } //
Check Jupyter theme if (body.getAttribute('data-jp-
theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') {
return 'light'; } // Guess based on a parent element's
```

color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* \s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s* \s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [20]:
```
y_pred = model.predict(X_test)
```

In [21]:
```
print("Predictions:", y_pred[:10])
print("Actual answers:", y_test.iloc[:10].
```

```
Predictions: [0 0 0 1 1 1 1 0 1 1]
Actual answers: [1, 0, 0, 1, 1, 1, 1, 0, 1,
1]
```

In [22]:
```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Test accuracy:", accuracy)
print("Test accuracy (%):", accuracy * 100
```

```
Test accuracy: 0.8059701492537313
Test accuracy (%): 80.59701492537313
```

In [23]:
```
results = X_test.copy()
results["Actual"] = y_test
results["Predicted"] = y_pred
results["Wrong"] = results["Actual"] != re

results.head()
```

Out[23]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Pclass</th> <th>Age</th> <th>Sex</th> <th>Fare</th> <th>Actual</th> <th>Predicted</th> <th>Wrong</th> </thead> <tbody> <th>709</th> <th>439</th> <th>840</th> <th>720</th> <th>39</th> </tbody> <tbody></tbody>

3	28.0	0	15.2458	1	0	True
2	31.0	0	10.5000	0	0	False
3	20.0	0	7.9250	0	0	False
2	6.0	1	33.0000	1	1	False
3	14.0	1	11.2417	1	1	False

In [24]:
```
results.groupby("Sex")["Wrong"].agg(["sum"
```

Out[24]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>sum</th> <th>count</th> <th>mean</th> <th>Sex</th> <th> </th> <th></th> <th></th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

34	167	0.203593
18	101	0.178218

In [25]:
```
results.groupby("Pclass")["Wrong"].agg(["s
```

Out[25]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>sum</th> <th>count</th> <th>mean</th> <th>Pclass</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody> </tbody>

20	77	0.259740
7	52	0.134615
25	139	0.179856

<p>I trained the shallow decision tree classifier to predict titanic passenger survial useing passanger class ,age,sexand fare. I filled missing ages with the median(28) ,i filled the age with 0 and femmal with

1.i split the data 70% tranning and 30% testing .I got 80.60% accuracy.The model had the highyest error rate for first class(25.97%) and made a highter percentage of error for male passengers(20.36%)than femmale passengers(17.82%)</p>

In [ ]: